

FACULTAD DE INGENIERIA

INGENIERIA DE SISTEMAS COMPUTACIONALES

FUNDAMENTOS DE PROGRAMACIÓN

INTEGRANTES:

Guiop Tucto, Abner Jhonny

Muñante Padilla, Rodolfo

Quispe Medina, David Isaac

Vargas Campián, Rubén Darío

DOCENTE:

Pardo Zapata, Percy Angel

2026

1. Observación de Campo y Bitácora del Proceso (Mesa de Partes Virtual)

Selección de identidad: Para este trabajo se seleccionó la **Mesa de Partes Virtual de la Municipalidad Distrital de San Juan de Lurigancho (MDSJL)**. Debido a los horarios laborales y con la autorización previa del docente, se optó por la modalidad virtual, la cual permite analizar de forma directa cómo una entidad pública gestiona la recepción digital de documentos bajo el marco de la Ley N.° 31170. Esta plataforma fue elegida porque presenta un flujo claro de registro, validación y confirmación que sirve como base para diseñar el prototipo del curso.

Proceso observado: El trámite realizado consistió en una Solicitud de Acceso a la información Pública dirigida a la Gerencia de Seguridad Ciudadana sobre el sistema de videovigilancia distrital. El flujo observado constó de tres etapas:

- **Ingreso de registro de datos:** Se accedió a la plataforma web institucional, se realizó el registro de usuario y se procedió con el trámite (tipo de servicio, tipo de documento, número correlativo, folios y asunto). Se adjuntó la solicitud formal en formato PDF.

- **Procesamiento y confirmación en línea:** Al enviar la solicitud, el sistema validó los campos, emitió un mensaje de confirmación inmediata en pantalla y generó el código temporal **SOL-2026-00001** con estado **PRESENTADO** fechado el 03/10/2026 a las 23:06 horas.

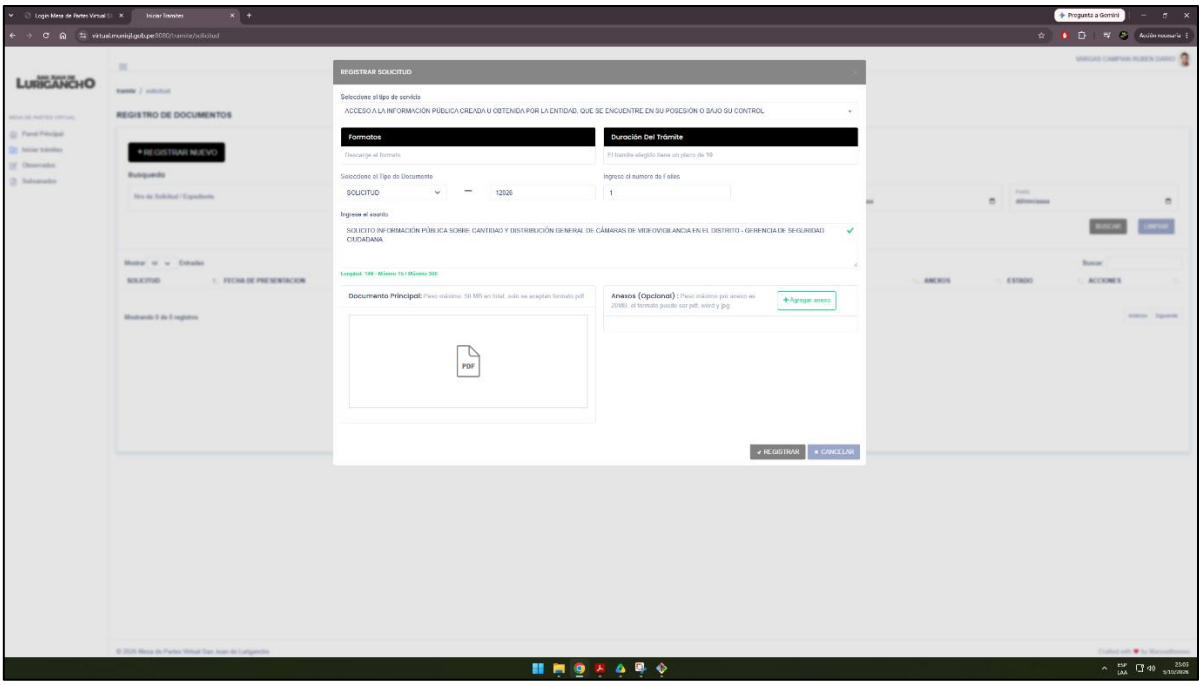
- **Validación administrativa y salida final:** El 05/10/2026 a las 08:02 horas (primer día hábil siguiente), la entidad revisó el documento y remitió una notificación formal por correo electrónico. El estado cambió a **APROBADO PARA SU ATENCIÓN** y se le asignó el número de expediente formal definitivo: **E-0049132-2026**.

A continuación de presentan las evidencias del proceso observado:

Figura 1. Registro del trámite y carga de requisitos

Se observa el llenado del formulario web con los datos de entrada requeridos y también se muestra el archivo PDF adjunto.

Fecha y hora de captura: 03/10/2026 – 23:05 h.



SOLICITUD N° 001-2026

SOLICITA: Información pública sobre el sistema de videovigilancia distrital.

A: Gerencia de Seguridad Ciudadana / Responsable de Acceso a la Información de la Municipalidad Distrital de San Juan de Lurigancho.

Yo, **Rubén Darío Vargas Campián,** identificado con DNI N° **72879693**, con domicilio en **Jr. Río Itaya Mz. M Lt. 29 Los Álamos San Juan de Lurigancho**, ante usted me presento y expongo:

Que, al amparo del Texto Único Ordenado de la Ley N.° 27806 (Ley de Transparencia y Acceso a la Información Pública), solicito se me brinde información pública sobre:

1. La cantidad total aproximada de cámaras de videovigilancia operativas bajo administración municipal en el distrito.
2. Las zonas o cuadrantes principales de cobertura del sistema de monitoreo.

Agradeceré remitir la respuesta al correo electrónico registrado en la plataforma.

San Juan de Lurigancho, 03/10/2026

Nombre: Ruben Darío Vargas Campián

DNI: 72879693

Figura 2. Confirmación inmediata y generación de código de solicitud

Ventana emergente que confirma el registro exitoso en la base de datos municipal y asigna el código SOL-2026-00001 en estado PRESENTADO

Fecha y hora de captura: 03/10/2026 – 23:09 h.

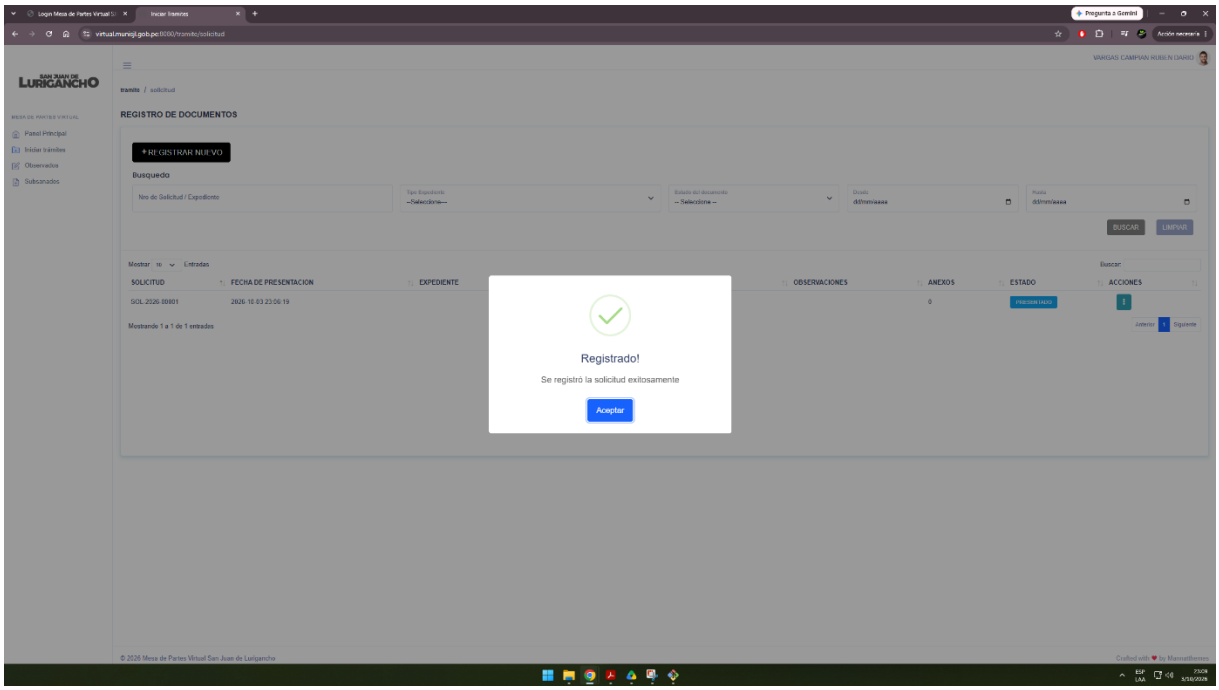
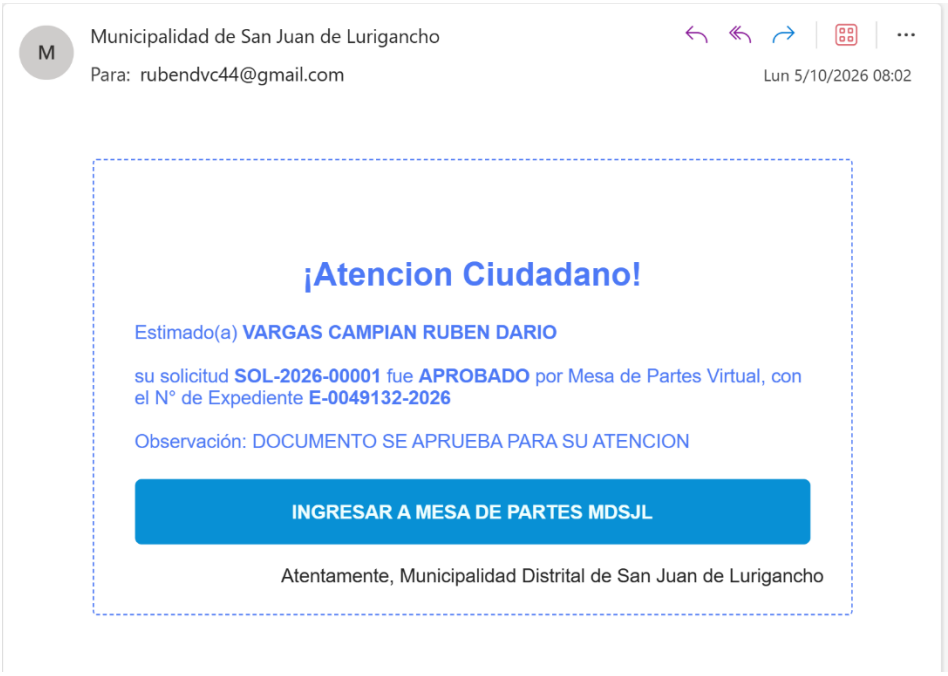


Figura 3. Notificación formal y asignación de expediente definitivo

Correo electrónico institucional confirmando la aprobación del trámite y asignando el número de expediente E-0049132-2026 para su atención interna.

Fecha y hora de captura: 05/10/2026 – 08:02 h.



2. Levantamiento de Requerimientos (Entrada, procesos y salida)

A partir de la experiencia con la plataforma virtual de la Municipalidad Distrital de San Juan de Lurigancho, se determinaron los componentes del sistema para el prototipo en Python:

- Tabla de Entradas, Procesos y Salidas

COMPONENTE	VARIABLE	DESCRIPCION
Entradas	tipo_servicio	Tipo de trámite seleccionado (ej. Acceso a la información pública)
	tipo_documento	Clasificación del escrito (ej. Solicitud)
	numero_documento	Número correlativo del documento (ej. 12026)
	folios	Cantidad de hojasdel archivo (>0)
	asunto	Detalle o motivo de la solicitud (no vacío)
	dni_solicitante	DNI del usuario (longitud exacta de 8 dígitos numéricos)
	nombres_apellidos	Nombre completo del ciudadano
	correo	Correo electrónico para el envío de notificaciones
	archivo_pdf	Documento principal adjunto en formato .pdf
Procesos	Validación	Verificar campos obligatorios y formato correcto de DNI.
	Marca de tiempo	Registro de la fecha y hora exacta del envío.
	Persistencia	Guardado de los registros en un archivo local para no perder datos.
Salidas	codigo_solicitud	Código inicial generado (ej. SOL-2026-00001)
	numero_expediente	Código formal asignado tras la revisión (ej. E-0049132-2026)
	estado	Estado actual (PRESENTADO, APROBADO, etc)
	notificacion	Mensaje de confirmación y resumen del registro

- Problemas frecuentes identificados y soluciones para el prototipo

    - Restricción en campos de texto: El formulario web no admitió caracteres especiales ni ceros a la izquierda (como guiones en 001-2026), forzando a escribir 12026.
      Solución: En el prototipo se usarán cadenas de texto (str) con validaciones flexibles para aceptar formatos administrativos estándar.

    - Latencia en la confirmación formal: Hubo una espera de casi dos días hasta recibir la confirmación por correo con el número de expediente formal definitivo.
      Solución: El programa emitirá un ticket de registro inmediato con su código correlativo de seguimiento en el momento de guardar los datos.

    - Validación de archivos: Si el usuario no adjunta el formato adecuado, el trámite se detiene.
      Solución: El algoritmo comprobará que la extensión del archivo sea válida antes de permitir el registro.

3. Generación y selección de soluciones

Se evalúan dos alternativas y se comparan frente a la posibilidad de tercerizar el software.

Alternativa 1: Modular con arreglos paralelos unidimensionales, esto permite mantener tipos nativos en cada arreglo. Con costo 0 ya que se desarrolla de manera interna en Python sin herramientas adicionales.

Alternativa 2: Matriz bidimensional de tamaño N x M donde cada fila es un expediente, esto obliga a forzar todos los datos a un solo tipo para llenar la matriz. De la misma manera tiene costo cero porque se desarrolla de manera interna.

Alternativa 3: Tercerizar con software comercial el cual tiene una configuración cerrada según lo que el proveedor nos venda, sin embargo, el precio es alto.

Dadas las alternativas se elije la numero 1 porque tiene correspondencia con el problema, cada registro comparte el mismo índice de posición, lo que facilita vincular los datos de un mismo tramite de manera intuitiva y sin correos de conversión. También facilita la implementación de los tres algoritmos clave pedidos: inserción, búsqueda y ordenamiento.

4. Diseño de algoritmo

Pseudocódigo del sistema simplificado de Mesa de Partes Digital implementado en PSeInt.

```
Algoritmo MesaDePartesSimple
    // Definición de tipos de variables
    Definir total, opc, i Como Entero
    Definir codigos, nombres, dnis, asuntos Como Cadena

    // Arreglos para almacenar hasta 30 registros
    Dimension codigos[30], nombres[30], dnis[30], asuntos[30]

    total ← 0
    opc ← 0

    Repetir
        Escribir ""
        Escribir "--- MENU PRINCIPAL ---"
        Escribir "1. Registrar expediente"
        Escribir "2. Listar expedientes"
        Escribir "3. Salir"
        Escribir "Elija una opcion: " Sin Saltar
        Leer opc

        Segun opc Hacer
            1:
                total ← total + 1
                codigos[total] ← "EXP-00" + ConvertirATexto(total)

                Escribir "Codigo asignado: ", codigos[total]
                Escribir "Ingrese DNI: " Sin Saltar
                Leer dnis[total]
                Escribir "Ingrese Nombre: " Sin Saltar
                Leer nombres[total]
                Escribir "Ingrese Asunto: " Sin Saltar
                Leer asuntos[total]

                Escribir "Expediente guardado con exito."

            2:
                Escribir ""
                Escribir "--- LISTADO DE EXPEDIENTES ---"
                Si total = 0 Entonces
                    Escribir "No hay registros cargados."
                SiNo
                    Para i ← 1 Hasta total Con Paso 1 Hacer
                        Escribir "Cod: ", codigos[i], " | DNI: ", dnis[i], " | Nombre: ", nombres[i], " | Asunto: ", asuntos[i]
                    FinPara
                FinSi

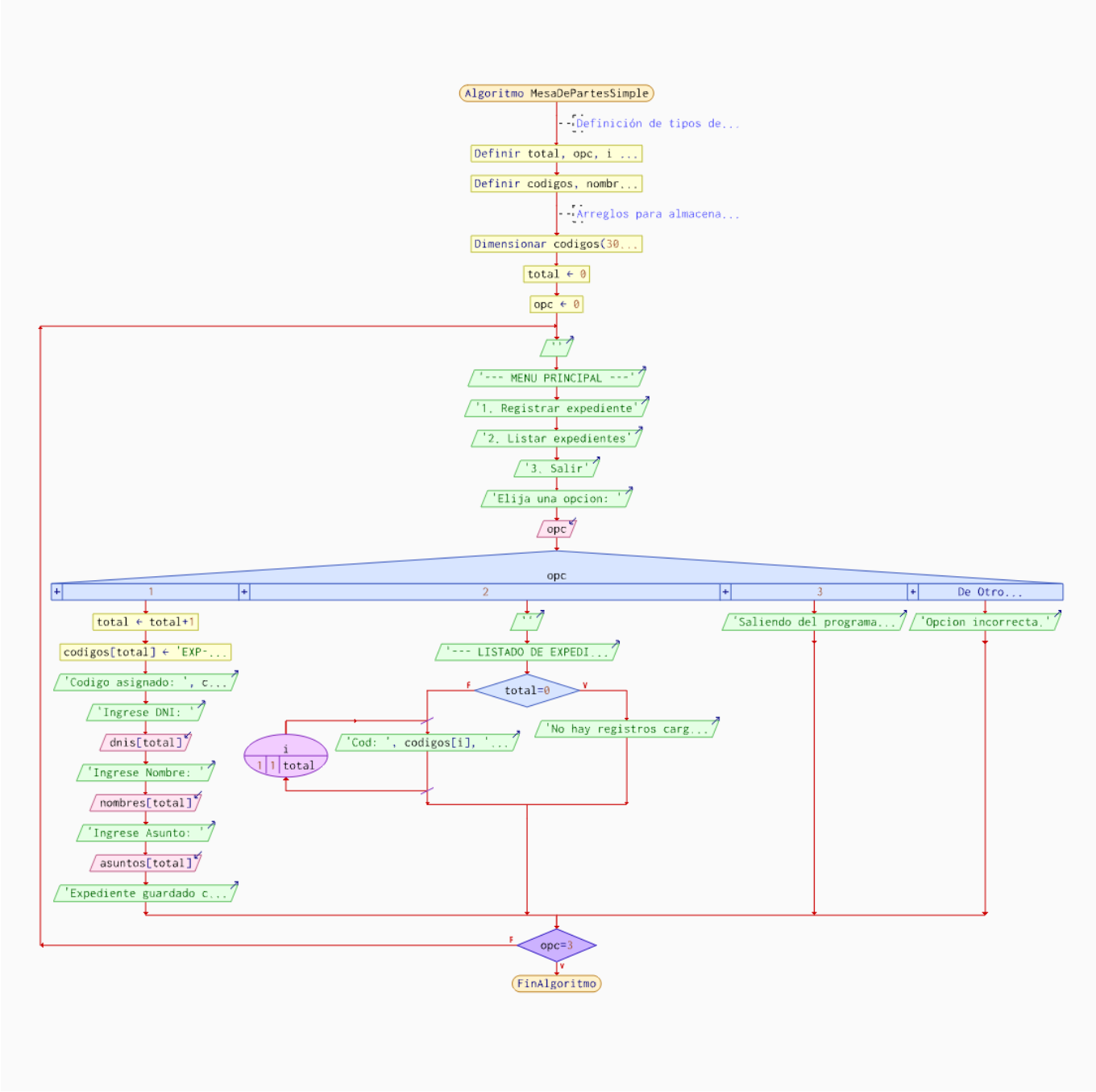
            3:
                Escribir "Saliendo del programa..."

            De Otro Modo:
                Escribir "Opcion incorrecta."
        FinSegun

    Hasta Que opc = 3
FinAlgoritmo

```

Diagrama de flujo del sistema de registro y listado de expedientes.



5. Implementación versionada

Código 1. Implementación modular del sistema de Mesa de Partes en Python.

```
codigos = []
dnis = []
nombres = []
apellidos = []
correos = []
telefonos = []
asuntos = []
descripciones = []

# Constantes del menú
OPCION_REGISTRAR = 1
OPCION_LISTAR = 2
OPCION_SALIR = 3


def mostrar_titulo(texto):
    """Genera un encabezado visual estandarizado."""
    print()
    print("=" * 50)
    print(texto.center(50))
    print("=" * 50)


def mostrar_menu():
    """Muestra el menú de opciones en la consola."""
    mostrar_titulo("MESA DE PARTES DIGITAL")
    print(f"    {OPCION_REGISTRAR}. Registrar expediente")
    print(f"    {OPCION_LISTAR}. Listar expedientes")
    print(f"    {OPCION_SALIR}. Salir")
    print("-" * 50)


def solicitar_texto(mensaje):
    """Solicita un texto obligatorio y valida que no esté vacío."""
    while True:
        texto = input(mensaje).strip()
        if not texto:
            print("Error: Este campo no puede quedar vacío.")
        else:
            return texto


def validar_dni(dni):
    """Valida que el DNI tenga exactamente 8 dígitos numéricos."""
    if len(dni) != 8:
```

```
        return False
    for c in dni:
        if not c.isdigit():
            return False
    return True


def validar_telefono(tel):
    """Valida que el teléfono tenga exactamente 9 dígitos numéricos."""
    if len(tel) != 9:
        return False
    for c in tel:
        if not c.isdigit():
            return False
    return True


def registrar_expediente():
    """Solicita, valida y almacena los datos de un nuevo expediente."""
    mostrar_titulo("REGISTRAR NUEVO EXPEDIENTE")

    # Generar código correlativo automático (EXP-001, EXP-002, etc.)
    nuevo_codigo = f"EXP-{len(codigos) + 1:03d}"
    print(f"Código asignado: {nuevo_codigo}")

    # Validar DNI
    while True:
        dni = solicitar_texto("Ingrese el DNI (8 dígitos): ")
        if validar_dni(dni):
            break
        print("Error: El DNI debe contener exactamente 8 números.")

    nombres_ingresados = solicitar_texto("Ingrese los nombres: ")
    apellidos_ingresados = solicitar_texto("Ingrese los apellidos: ")
    correo = solicitar_texto("Ingrese el correo electrónico: ")

    # Validar teléfono
    while True:
        tel = solicitar_texto("Ingrese el teléfono (9 dígitos): ")
        if validar_telefono(tel):
            break
        print("Error: El teléfono debe contener exactamente 9 números.")

    # Flujo directo al trámite (sin la pregunta redundante)
    asunto = solicitar_texto("Ingrese el asunto del trámite: ")
    descripcion = solicitar_texto("Ingrese la descripción detallada: ")

    # Almacenar en las listas globales
    codigos.append(nuevo_codigo)
```

```
    dnis.append(dni)
    nombres.append(nombres_ingresados)
    apellidos.append(apellidos_ingresados)
    correos.append(correo)
    telefonos.append(tel)
    asuntos.append(asunto)
    descripciones.append(descripcion)

    print(f"\n¡Éxito! Expediente {nuevo_codigo} registrado correctamente.")


def listar_expedientes():
    """Muestra todos los expedientes registrados."""
    mostrar_titulo("LISTADO DE EXPEDIENTES")

    if not codigos:
        print("No hay expedientes registrados en el sistema.")
        return

    for i in range(len(codigos)):
        print(f"Expediente N° {i + 1} | Código: {codigos[i]}")
        print(f"  Ciudadano: {nombres[i]} {apellidos[i]}")
        print(f"  Identificación (DNI): {dnis[i]}")
        print(f"  Contacto: Tel: {telefonos[i]} | Correo: {correos[i]}")
        print(f"  Asunto: {asuntos[i]}")
        print(f"  Descripción: {descripciones[i]}")
        print("-" * 50)


def ejecutar_sistema():
    """Controla el flujo principal del programa mediante el menú."""
    while True:
        mostrar_menu()
        opcion = input("Seleccione una opción del menú: ").strip()

        if opcion == str(OPCION_REGISTRAR):
            registrar_expediente()
        elif opcion == str(OPCION_LISTAR):
            listar_expedientes()
        elif opcion == str(OPCION_SALIR):
            print("\nGracias por usar la Mesa de Partes Digital. ¡Hasta
luego!")
            break
        else:
            print("Error: Opción no válida. Por favor, intente de nuevo.")


ejecutar_sistema()
```

6. Prueba de datos reales y reflexión

Se ejecutaron pruebas en *VS Code* utilizando los datos de la solicitud real presentada en Mesa de Partes Virtual de la Municipalidad Distrital de San Juan de Lurigancho, evaluando casos de control y casos normales.

```
================================================
             MESA DE PARTES DIGITAL
================================================
    1. Registrar expediente
    2. Listar expedientes
    3. Salir
------------------------------------------------
Seleccione una opción del menú: 1

================================================
           REGISTRAR NUEVO EXPEDIENTE
================================================
Código asignado: EXP-001
Ingrese el DNI (8 dígitos): 72879693
Ingrese los nombres: Ruben
Ingrese los apellidos: Vargas Campian
Ingrese el correo electrónico: rubenvargas@gmail.com
Ingrese el teléfono (9 dígitos): 973473713
Ingrese el asunto del trámite: Camaras
Ingrese la descripción detallada: Solicitud de cantidad de camaras

¡Éxito! Expediente EXP-001 registrado correctamente.

================================================
             MESA DE PARTES DIGITAL
================================================
    1. Registrar expediente
    2. Listar expedientes
    3. Salir
------------------------------------------------
Seleccione una opción del menú:
```

Se muestran resultados en caso el usuario cometa errores

```
================================================
             MESA DE PARTES DIGITAL
================================================
    1. Registrar expediente
    2. Listar expedientes
    3. Salir
------------------------------------------------
Seleccione una opción del menú: 2

================================================
             LISTADO DE EXPEDIENTES
================================================
No hay expedientes registrados en el sistema.

================================================
             MESA DE PARTES DIGITAL
================================================
    1. Registrar expedientes
    2. Listar expedientes
    3. Salir
------------------------------------------------
Seleccione una opción del menú: 1

================================================
           REGISTRAR NUEVO EXPEDIENTE
================================================
Código asignado: EXP-001
Ingrese el DNI (8 dígitos): 7287969x
Error: El DNI debe contener exactamente 8 números.
Ingrese el DNI (8 dígitos): 72879693
Ingrese los nombres: Ruben
Ingrese los apellidos: Vargas
Ingrese el correo electrónico: ruben@gmail.com
Ingrese el teléfono (9 dígitos): 97347371
Error: El teléfono debe contener exactamente 9 números.
Ingrese el teléfono (9 dígitos): 97347371x
Error: El teléfono debe contener exactamente 9 números.
Ingrese el teléfono (9 dígitos): 973473713
Ingrese el asunto del trámite: Videovigilania
Ingrese la descripción detallada: conteo de camaras

¡Éxito! Expediente EXP-001 registrado correctamente.
```

Reflexión

La implementación de rutinas modulares como solicitar_texto(), validar_dni() y validar_telefono() resuelve la problemática identificada en la plataforma municipal respecto a entradas mal formateadas o campos en blanco. El uso de .strip() y validaciones carácter por carácter (c.isdigit()) evita inconsistencias antes de la inserción en memoria.

Se investigó e implementó la especificación de formato de cadenas en Python

```
nuevo_codigo = f”EXP-{len(códigos) + 1:03d}”
```

Esto permite que números enteros como 1, 2 o 15 se completen automáticamente con ceros a la izquierda (EXP-001, EXP-002, EXP-015), resolviendo directamente la restricción de formatos que se observó en el formulario web de la municipalidad.

Enlace de GitHub

https://github.com/DavidIsaacQ/Grupo3_Semana_10